



Experiment No.: 4

Title: Functional Testing using Selenium



Aim: To perform functional testing using Selenium.

Resources needed: Internet, Firefox Browser, and Selenium IDE

Theory:

Selenium is a portable software-testing framework for web applications. Selenium provides a playback (formerly also recording) tool for authoring tests without the need to learn a test scripting language (Selenium IDE). It also provides a test domain-specific language (Selenese) to write tests in a number of popular programming languages, including C#, Groovy, Java, Perl, PHP, Python, Ruby and Scala. The tests can then run against most modern web browsers. Selenium deploys on Windows, Linux, and macOS platforms. It is open-source software, released under the Apache 2.0 license. Web developers can download and use it without charge.

Selenium IDE is an integrated development environment for Selenium scripts. It is implemented as a Firefox extension, and allows you to record, edit, and debug tests. Selenium IDE includes the entire Selenium Core, allowing you to easily and quickly record and play back tests in the actual environment that they will run in.

Selenium IDE is not only a recording tool: it is a complete IDE. You can choose to use its recording capability, or you may edit your scripts by hand. With auto complete support and the ability to move commands around quickly, Selenium IDE is the ideal environment for creating Selenium tests no matter what style of tests you prefer.

Features:

- Easy record and playback
 - Intelligent field selection will use IDs, names, or XPath as needed
 - Autocomplete for all common Selenium commands
 - Walk through tests
 - Debug and set breakpoints
 - Save tests as HTML, Ruby scripts, or any other format
 - Support for Selenium [user-extensions.js](#) file
 - Option to automatically assert the title of every page
-

Procedure:

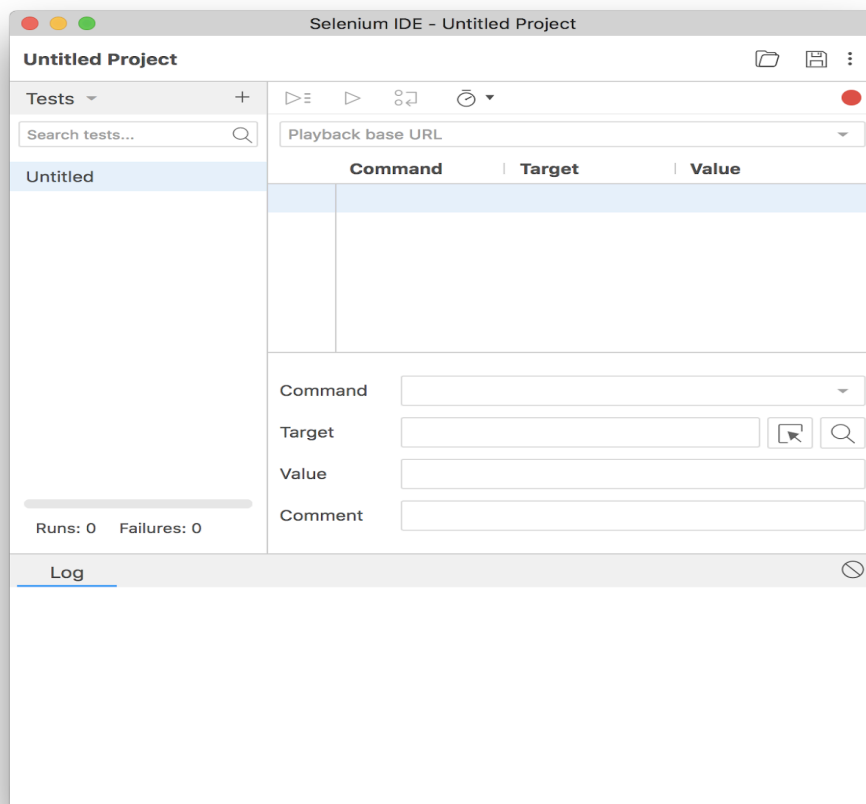
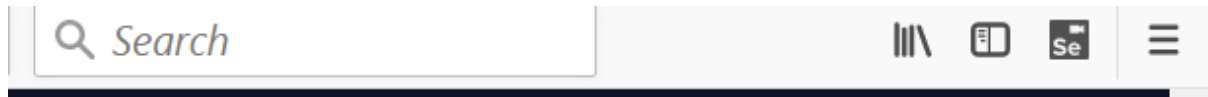
1. Install Selenium IDE.

Step1 - Launch Firefox and navigate to

<https://addons.mozilla.org/en-US/firefox/addon/selenium-ide/>. Click on Add to Firefox.

Step2 - Click on Add Button to Give Permission.

Step3 - Launch Selenium IDE Click on Selenium Icon appear on the right hand side of the address bar.



2. Open any Website (under test) on Firefox -

For example: <http://automationpractice.com/index.php>

3. Document and Implement the following Features:

1. Auto Scripting and Playback
2. Verification
3. Assertion
4. Store Variables

Note: Since Selenium IDE is Firefox based extension we need to Install Firefox

<https://www.mozilla.org/en-US/firefox/57.0/releasenotes/>

Results: Snapshots of Script and log

 Extension: (Selenium IDE) - Selenium IDE - STQA_Experiment-4_Fun...—□×

Project: STQA_Experiment-4_FunctionalTest*📁📁💾⋮

Tests ▾+

Search tests... 🔍

Search Test*

▶▶▶▶⌚▼🔗⏸REC

https://www.wikipedia.org/ ▼

	Command	Target	Value
1	open	https://www.wikipedi a.org/	
2	set window size	1295x687	
3	click	id=searchInput	
4	type	id=searchInput	Software testing
5	click	css=.active > .suggest ion-text	

Command

Target

Value

Description

⏸

🔗

🔗

🔍

Log

Reference

🔕

 Extension: (Selenium IDE) - Selenium IDE - STQA_Experiment-4_Fun...—□×

Project: STQA_Experiment-4_FunctionalTest*📁📁💾⋮

Tests ▾+

Search tests... 🔍

Search Test*

▶▶▶▶⌂⌚▼

↶⏸Ⓡ

	Command	Target	Value
1	open	https://www.wikipedi a.org/	
2	set window size	1295x687	
3	click	id=searchInput	
4	type	id=searchInput	Software testing
5	click	css=.active > .suggest ion-text	
6	store text	id=firstHeading	articleTitle

Command

store text ▾

//



Target

id=firstHeading





Value

articleTitle

Description

Log

Reference

🔒

 Extension: (Selenium IDE) - Selenium IDE - STQA_Experiment-4_Fun...—□×

Project: STQA_Experiment-4_FunctionalTest*📁📁💾⋮

Tests ▾+

🔍

Search Test*





	Command	Target	Value
1	open	https://www.wikipedi a.org/	
2	set window size	1295x687	
3	click	id=searchInput	
4	type	id=searchInput	Software testing
5	click	css=.active > .suggest ion-text	
6	store text	id=firstHeading	articleTitle
7	echo	\${articleTitle}	

Command

echo ▾//

Target

\${articleTitle}  

Value

Description

Log

Reference



 Extension: (Selenium IDE) - Selenium IDE - STQA_Experiment-4_Fun...—□×

Project: STQA_Experiment-4_FunctionalTest*📁📁💾⋮

Tests ▾+

🔍

Search Test*





	Command	Target	Value
1	open	https://www.wikipedi a.org/	
2	set window size	1295x687	
3	click	id=searchInput	
4	type	id=searchInput	Software testing
5	click	css=.active > .suggest ion-text	
6	store text	id=firstHeading	articleTitle
7	echo	\${articleTitle}	

Command

echo ▾//

Target

\${articleTitle}  

Value

Description

Log

Reference



 Extension: (Selenium IDE) - Selenium IDE - STQA_Experiment-4_Fun...—□×

Project: STQA_Experiment-4_FunctionalTest*📁📁💾⋮

Tests ▾+

🔍

Search Test*





	Command	Target	Value
1	open	https://www.wikipedi a.org/	
2	set window size	1295x687	
3	click	id=searchInput	
4	type	id=searchInput	Software testing
5	click	css=.active > .suggest ion-text	
6	store text	id=firstHeading	articleTitle
7	echo	\${articleTitle}	
8	verify text	id=firstHeading	Software testing

Command

verify text ▾//

Target

id=firstHeading 

Value

Software testing

Description

Log

Reference



The screenshot displays the Selenium IDE interface within a Mozilla Firefox browser window. The project is named 'STQA_Experiment-4_FunctionalTest*'. The test script, titled 'Search Test', is loaded and shows a series of commands and their targets. The commands are: open, set window size, click, type, click, store text, echo, verify text, and assert title. The targets are: https://www.wikipedia.org/, 1295x687, id=searchinput, id=searchinput, css=active > suggestion-text, id=firstHeading, \${articleTitle}, id=firstHeading, and Software testing - Wikipedia. The values are: Software testing, articleTitle, and Software testing. The log at the bottom shows the execution of these commands, all of which completed successfully.

Command	Target	Value
open	https://www.wikipedia.org/	
set window size	1295x687	
click	id=searchinput	
type	id=searchinput	Software testing
click	css=active > suggestion-text	
store text	id=firstHeading	articleTitle
echo	\${articleTitle}	
verify text	id=firstHeading	Software testing
assert title	Software testing - Wikipedia	

Log Reference

Running "Search Test"

1. open on https://www.wikipedia.org/ OK 18:55:41

2. setWindowSize on 1295x687 OK 18:55:43

3. click on id=searchinput OK 18:55:43

4. type on id=searchinput with value Software testing OK 18:55:43

5. Trying to find css=active > suggestion-text... OK 18:55:48

6. storeText on id=firstHeading with value articleTitle OK 18:55:48

echo: Software testing 18:56:08

8. verifyText on id=firstHeading with value Software testing OK 18:56:14

9. assertTitle on Software testing - Wikipedia OK 18:56:15

Search Test completed successfully 18:56:15

Questions:

1. Explore one additional feature of Selenium not listed in the write up. Document and implement it.

Ans: An additional powerful feature of Selenium IDE not listed in the write-up is Control Flow Logic, which allows for creating more intelligent and adaptive tests using commands like **if**, **else**, **end**, and **while**.

- **Documentation:**

Simple record-and-playback scripts follow a linear path and will fail if the application's state is not exactly as expected. Control Flow Logic allows a test script to make decisions. For example, a test can check if an element is present on a page and perform a different set of actions based on the result. This is crucial for testing dynamic websites where content can change. The main commands are:

- **if / else if / else / end:** Executes a block of commands only if a certain condition is met.
- **while / end:** Repeats a block of commands as long as a condition is true.
- **times / end:** Repeats a block of commands a fixed number of times.

- **Implementation Example (Using if/end):**

Let's imagine a test scenario where we want to log out a user only if they are currently logged in. We can check for the presence of a "Sign out" button.

Website: <http://automationpractice.com/index.php>

Scenario:

1. Go to the website.
2. Check if the "Sign out" button is visible.
3. If it is visible, click it.
4. If it is not visible, the script does nothing and ends.

Selenium IDE Script:

Command	Target	Value
open	/index.php	
store element present	css=.logout	isLoggedIn
if	\${isLoggedIn}	
click	css=.logout	
end		

In this script:

1. We first check for the presence of an element with the CSS class .logout (the "Sign out" button) and store the result (true or false) in a variable named isLoggedIn.
2. The if command checks the value of the \${isLoggedIn} variable.
3. If the value is true, the script proceeds to click the "Sign out" button.
4. The end command marks the conclusion of the if block. If the value was false, the script would have skipped directly to the end command.

Outcomes: CO3 – Apply recent automation tools for testing software.

Conclusion: (Conclusion to be based on outcomes)

In conclusion, this experiment provided a practical introduction to automated functional testing using Selenium IDE. We successfully recorded, executed, and validated test cases, learning the crucial difference between verifications and assertions. The experiment confirms that Selenium IDE is an efficient and accessible tool for quickly automating browser tests to ensure web applications function as intended.

Grade: AA / AB / BB / BC / CC / CD / DD

Signature of faculty in-charge with date

Reference Websites:

1. https://www.seleniumhq.org/docs/02_selenium_ide.jsp
 2. <https://www.guru99.com/selenium-tutorial.html>
 3. <http://demo.guru99.com/test/newtours/index.php>
 4. <https://phptravels.com/demo/>
 5. <http://automationpractice.com/index.php>
-

